

State Machine & VHDL Coding

Module Convenor: Dr. T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

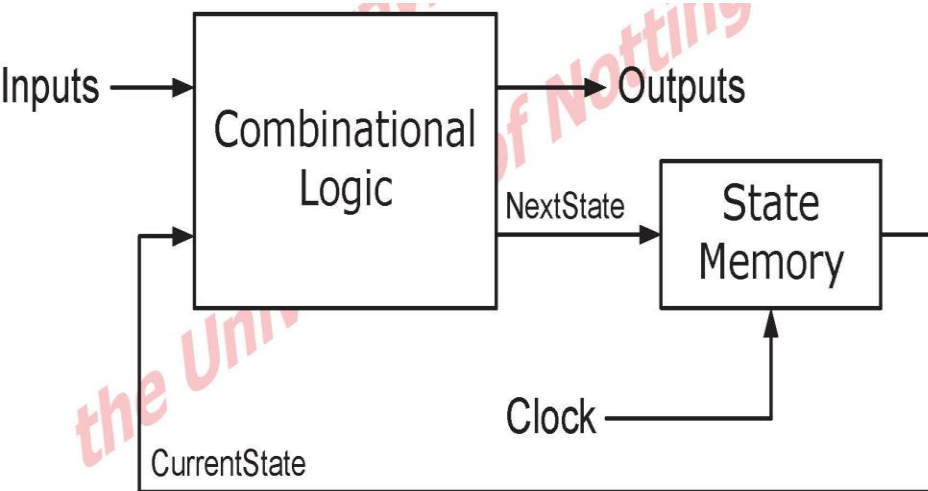
Department of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus

- Topics to be covered in this class
 - VHDL coding of Moore and Mealy Machines
 - Asynchronous state machine
 - Examples

Intro to State Machine Coding

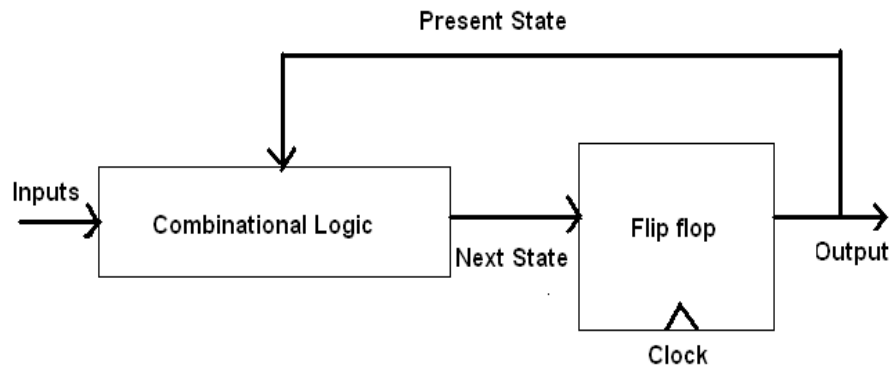
- State Machine => is a model of behavior composed of states, transitions and actions used to implement the control functions in an effective way.
- A state
 - stores information about the past, i.e. it reflects the input changes from the system start to the present moment.
- A transition
 - indicates a state change and is described by a condition that would need to be fulfilled to enable the transition.
- An action
 - is a description of an activity that is to be performed at a given moment
 - Entry action
 - execute the action *when entering* the state
 - Exit action
 - execute the action *when exiting* the state
 - Input action
 - execute the action dependent on present state and input conditions
 - Transition action
 - execute the action when performing a certain transition

Intro to State Machine Coding

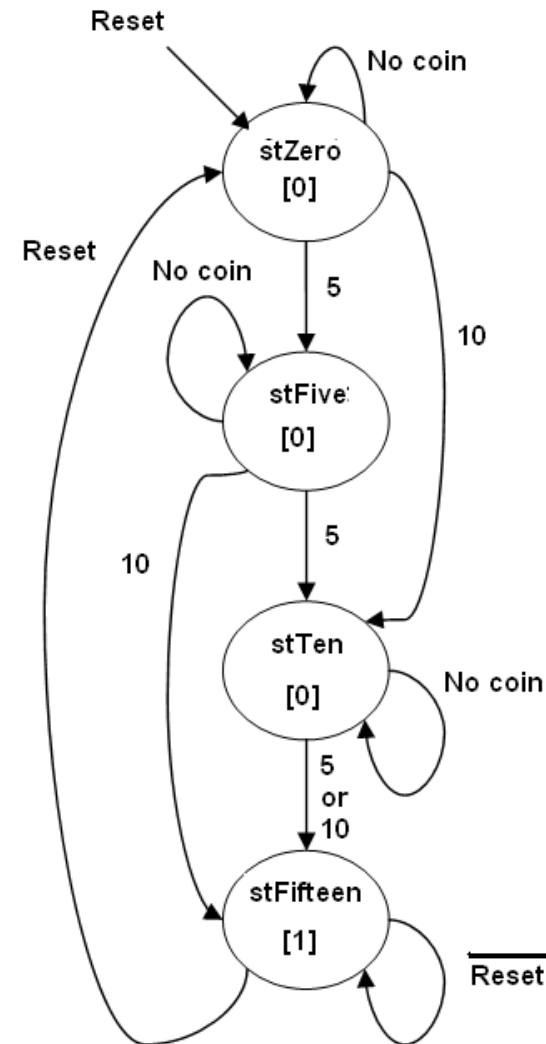


- The current state of the state machine is stored in the state memory
- The next state of the state machine is a function of the current state and the inputs
- From VHDL coding point of view, basically, an FSM consists of two processes
- One is a **synchronous** process, the other is an **asynchronous** process.
- Within the **synchronous** process, the next state will become the current state after a clock cycle (either after the rising edge of the clock signal or after the falling edge of the clock signal)
- Within the **asynchronous** process, the next state is determined based on the current state and the inputs

Moore Machine



- Out is the function of the “present state” only.
- The output cannot change its value immediately after a change at the inputs.
- Hence the output changes synchronously with Clock and state transition.
- Now let us consider a vending machine
- Assume:
 - Only chocolates (15 cents each) are available in this vending machine
 - This vending machine only accepts 5 cent and 10 cent coins
 - No change will be given if you insert two 10 cent coins

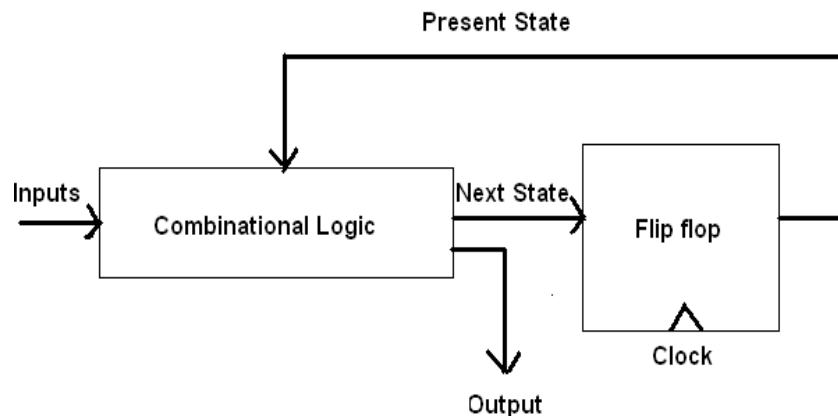


Moore Machine

```
library ieee;
use ieee.std_logic_1164.all;
entity VendingMachine is
    port(
        Reset, Clock, Ten, Five :in std_logic;
        Chocolate: out std_logic);
end VendingMachine;
architecture Arch of VendingMachine is
    --state type declaration
    type StateType is (stZero, stFive, stTen, stFifteen);
    --state declaration
    signal PresentState: StateType;
    signal NextState: StateType;
begin
    Sync_Proc: process (Reset, Clock)
    begin
        if Clock' event and Clock = '1' then
            if Reset = '1' then
                PresentState <= stZero;
                NextState <= stZero
            else
                PresentState <= NextState;
            end if;
        end if;
    end process;
    Async_Proc: process (PresentState , Five, Ten)
    begin
        --signal defaults
        Chocolate <= '0';
```

```
        case PresentState is
            when stZero =>
                if Five = '1' then
                    NextState<= stFive;
                elsif Ten= '1' then
                    NextState<= stTen;
                else
                    NextState<= stZero;
                end if;
            when stFive =>
                if Five = '1' then
                    NextState <= stTen;
                elsif Ten = '1' then
                    NextState <= stFifteen;
                else
                    NextState <= stFive;
                end if;
            when stTen =>
                if Five = '1' or Ten = '1' then
                    NextState <= stFifteen;
                else
                    NextState <= stTen;
                end if;
            when stFifteen =>
                Chocolate <= '1';
                NextState<= stZero;
            when others =>
                NextState<= stZero;
        end case;
    end process;
end Arch;
```

Mealy Machine

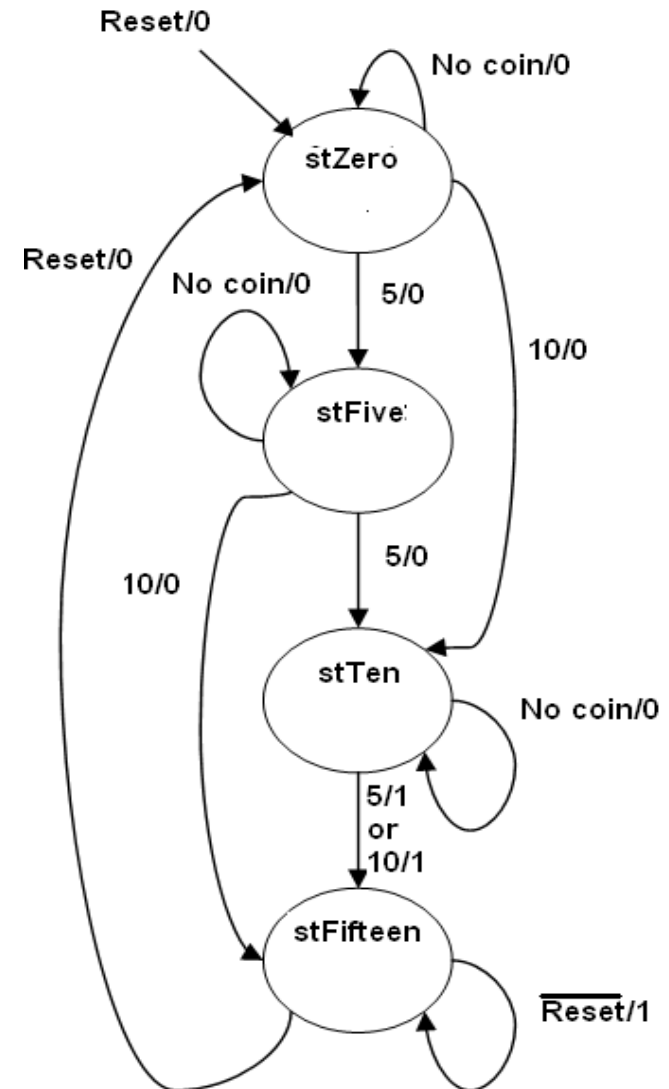


The output of the Mealy Machine is a function of the present state and all the inputs

The output can change its value immediately after a change at the inputs.

Hence the output is independent of Clock or asynchronous to the clock signal.

The inputs must be GLITCH free.





Mealy Machine

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity VendingMachine is
5     port(Reset, Clock, Ten, Five :in std_logic;
6         Chocolate: out std_logic);
7 end VendingMachine;
8
9 architecture Arch of VendingMachine is
10 --state type declaration
11 type StateType is (stZero, stFive, stTen, stFifteen);
12 --state declaration
13 signal PresentState: StateType;
14 signal NextState: StateType;
15 begin
16     process (Reset, Clock)
17     begin
18         if Reset = '1' then
19             PresentState <= stZero;
20             NextState <= stZero;
21
22         elsif Clock'event and Clock = '1' then
23             case PresentState is
24                 when stZero =>
25                     if Five = '1' then
26                         NextState<= stFive;
27                     elsif Ten= '1' then
28                         NextState<= stTen;
29                     else
30                         NextState<= stZero;
31                     end if;
32                 when stFive =>
33                     if Five = '1' then
34                         NextState <= stTen;
35                     elsif Ten = '1' then
36                         NextState <= stFifteen;
37                     else
38                         NextState <= stFive;
39                     end if;
40
```

```
40         when stTen =>
41             if Five = '1' or Ten = '1' then
42                 NextState <= stFifteen;
43             else
44                 NextState <= stTen;
45             end if;
46         when stFifteen =>
47             NextState<= stZero;
48         when others =>
49             NextState<= stZero;
50     end case;
51     PresentState <= NextState;
52 end if;
53 end process;
54 process (PresentState, Ten, Five)
55 begin
56     case PresentState is
57         when stZero =>
58             Chocolate <= '0';
59         when stFive =>
60             if Ten = '1' then
61                 Chocolate <= '1';
62             else
63                 Chocolate <= '0';
64             end if;
65         when stTen =>
66             if Five = '1' or Ten = '1' then
67                 Chocolate <= '1';
68             else
69                 Chocolate <= '0';
70             end if;
71         when stFifteen =>
72             Chocolate <= '1';
73         when others =>
74             Chocolate <= '0';
75     end case;
76 end process;
77 end Arch;
```



TestBench

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
ENTITY testbench IS
END testbench;

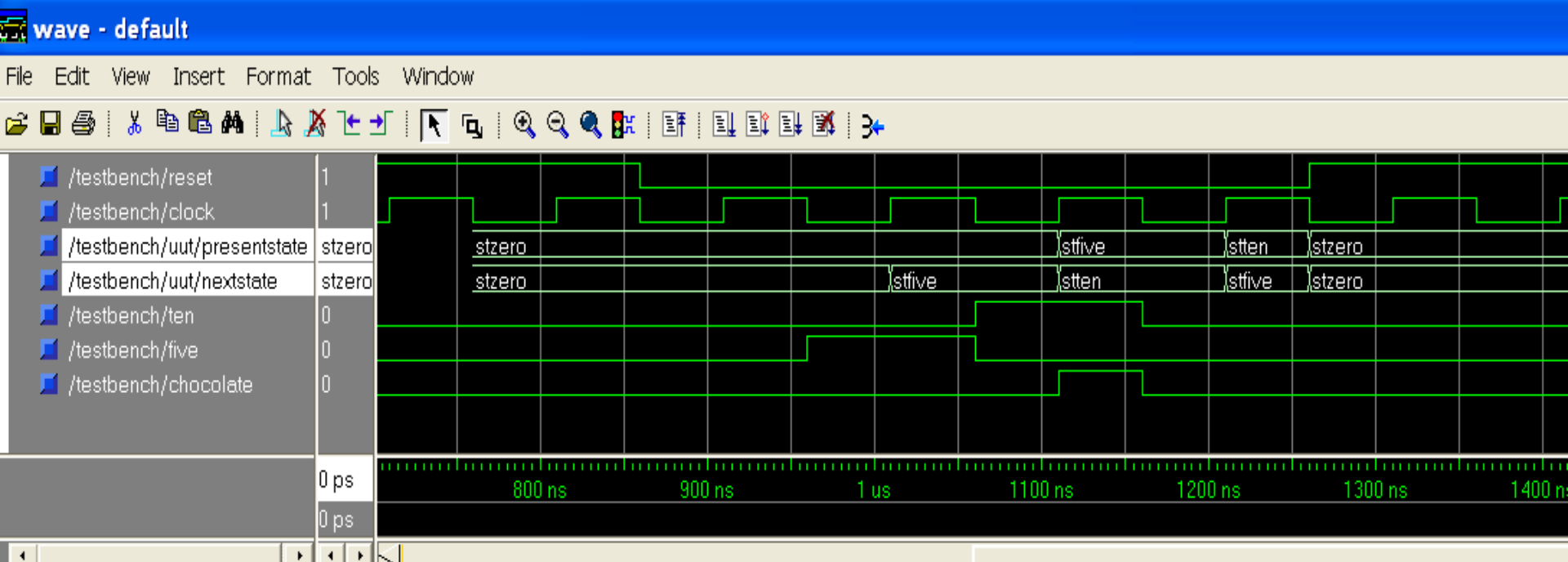
ARCHITECTURE behavior OF testbench IS
    COMPONENT vendingmachine
    PORT(
        reset : IN std_logic;
        clock : IN std_logic;
        ten : IN std_logic;
        five : IN std_logic;
        chocolate : OUT std_logic
    );
    END COMPONENT;

    SIGNAL reset : std_logic;
    SIGNAL clock : std_logic;
    SIGNAL ten : std_logic;
    SIGNAL five : std_logic;
    SIGNAL chocolate : std_logic;

BEGIN
    uut: vendingmachine PORT MAP(
        reset => reset,
        clock => clock,
        ten => ten,
        five => five,
        chocolate => chocolate
    );

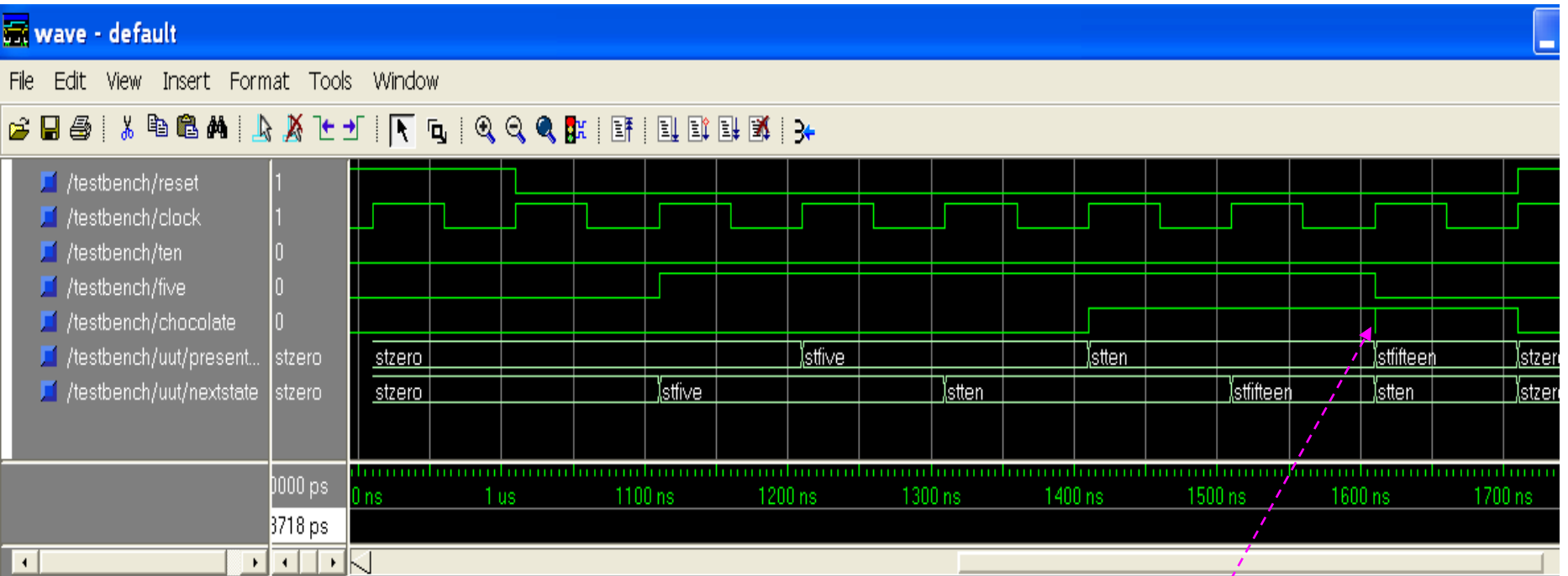
-- *** Test Bench - User Defined Section ***
tb : PROCESS
BEGIN
    wait; -- will wait forever
END PROCESS;
-- *** End Test Bench - User Defined Section ***
END behavior;
```

Simulation



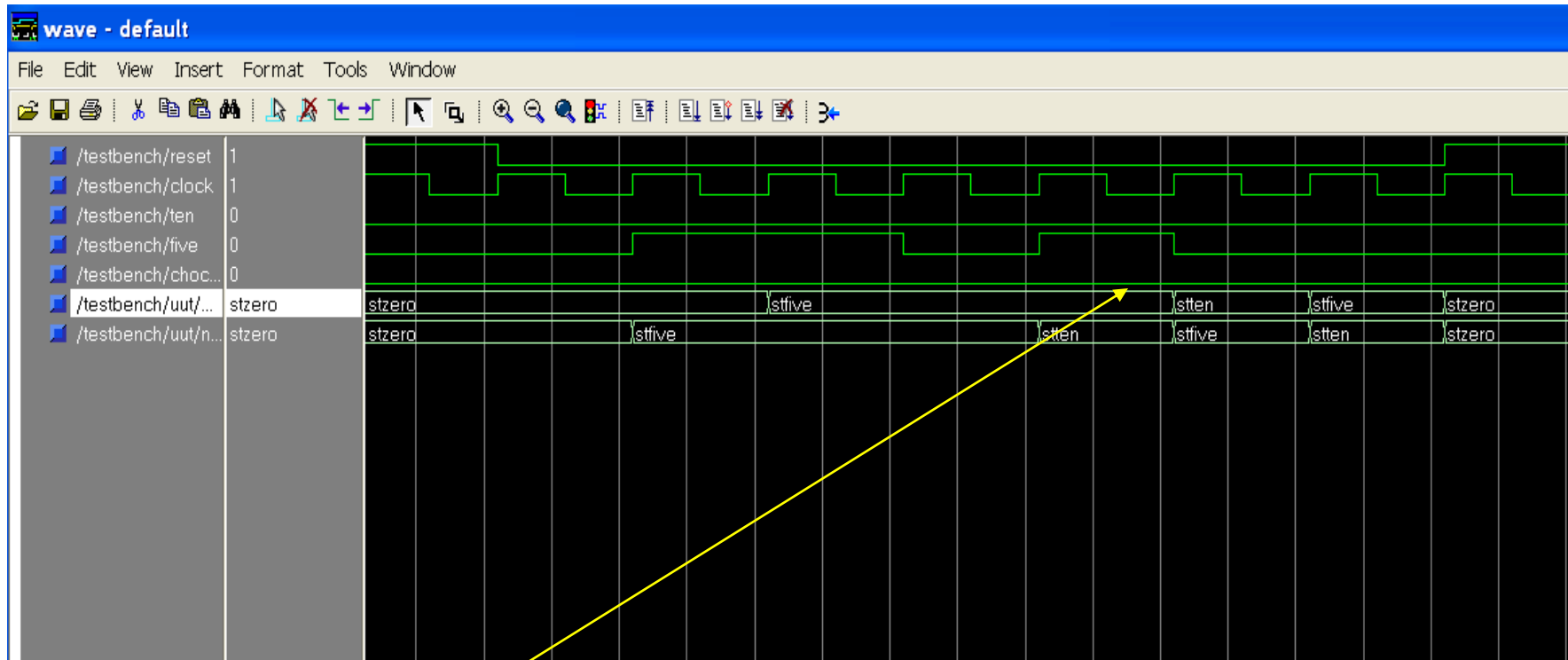
5 cent + 10 cent

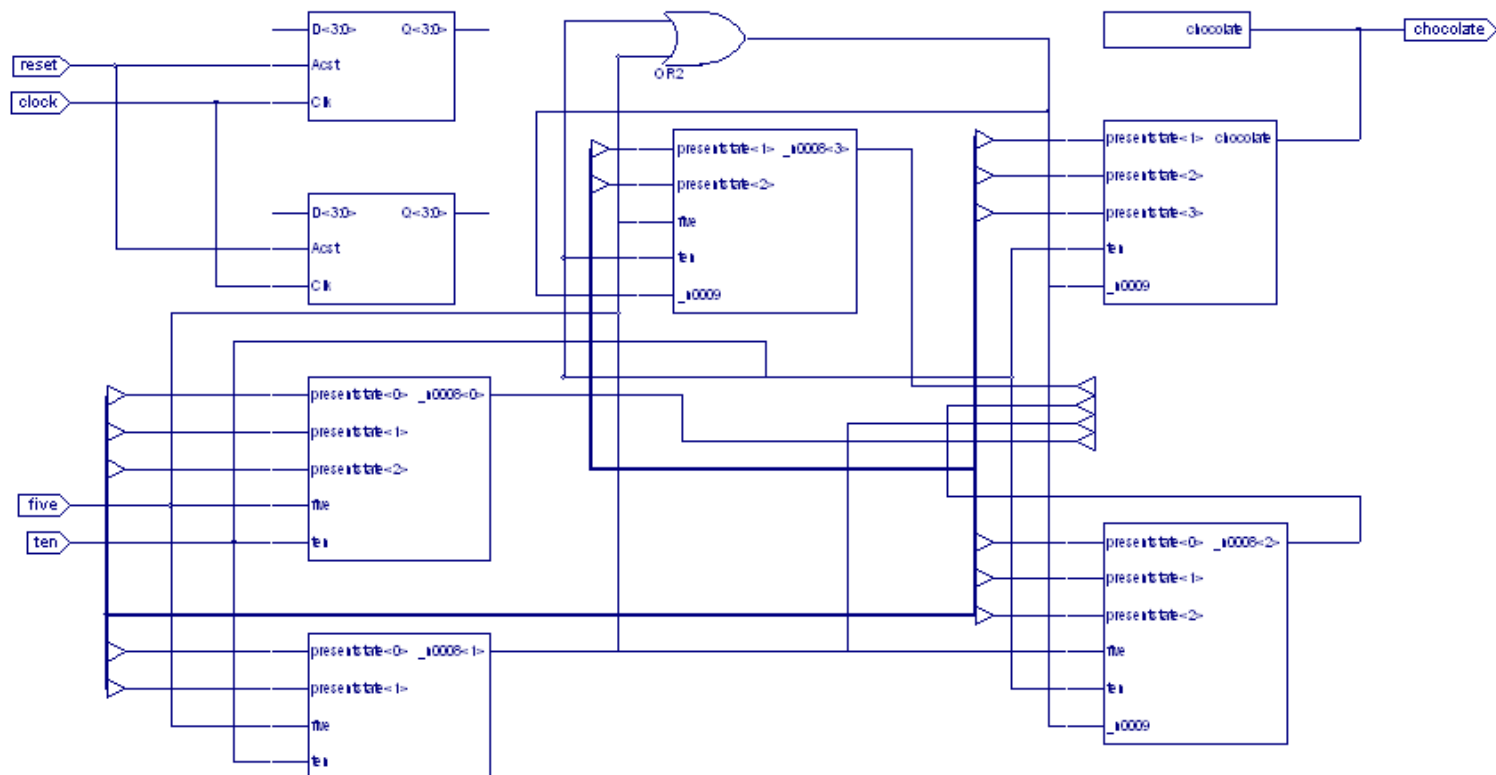
Simulation



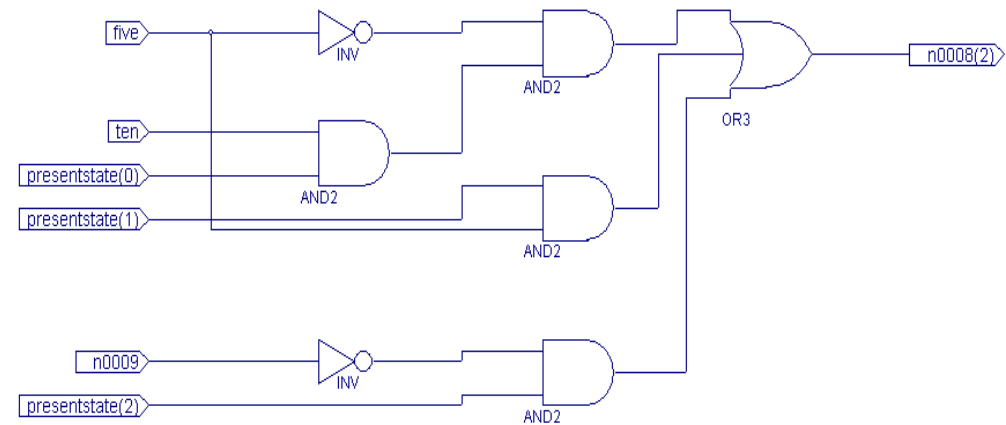
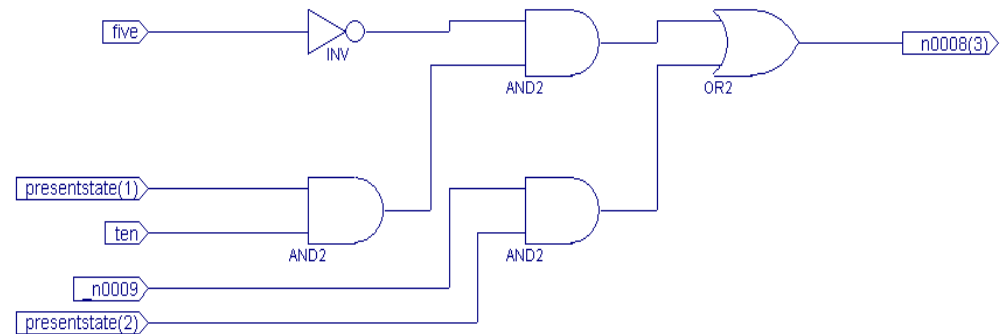
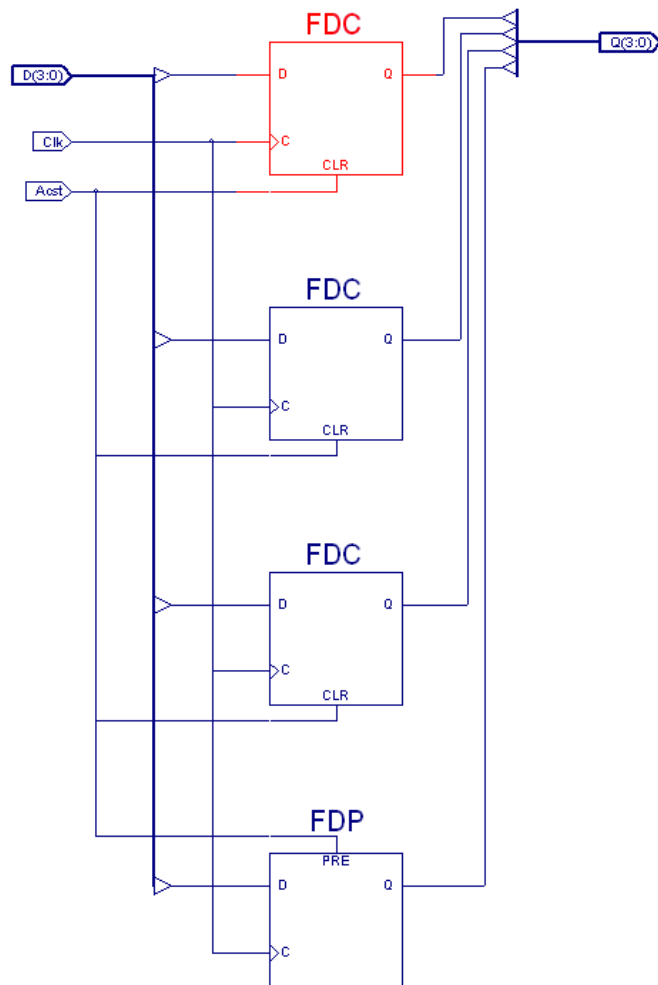
5 cent + 5 cent + 5 cent

Glitch





Gate Level



Attributes

- Enumeration type , such as stZero, stFive, stTen and stFifteen.
- Control the final state encoding by using attribute declaration and attribute specification.
- If we want the four states to be encoded as “00”, “01”, “11” and “10”, we can change the state declaration by adding the attribute declaration and attribute specification to the architecture declarative region.

--attribute declaration

attribute enum_encoding : string;

--state type declaration

type StateType **is** (stZero, stFive, stTen, stFifteen);

--attribute specification

attribute enum_encoding **of** StateType: **type** is “00 01 11 10”;

--state declaration

signal CurrentState: StateType;

signal NextState: StateType;

Observe:

This is known as GrayFSM encoding, which guarantees that only one state variable switches between two consecutive states
‘enum_encoding’ is a Xilinx synthesis constraint, which controls FSM encoding

Attributes

We can have another state encoding as follows:

--attribute declaration

attribute enum_encoding : string;

--state type declaration

type StateType **is** (stZero, stFive, stTen, stFifteen);

--attribute specification

attribute enum_encoding **of** StateType: **type is** "0001 0010 0100
1000";

--state declaration

signal CurrentState: StateType;

signal NextState: StateType;

Observe:

- The above is called as one-hot encoding, because only one bit is logic one in each state

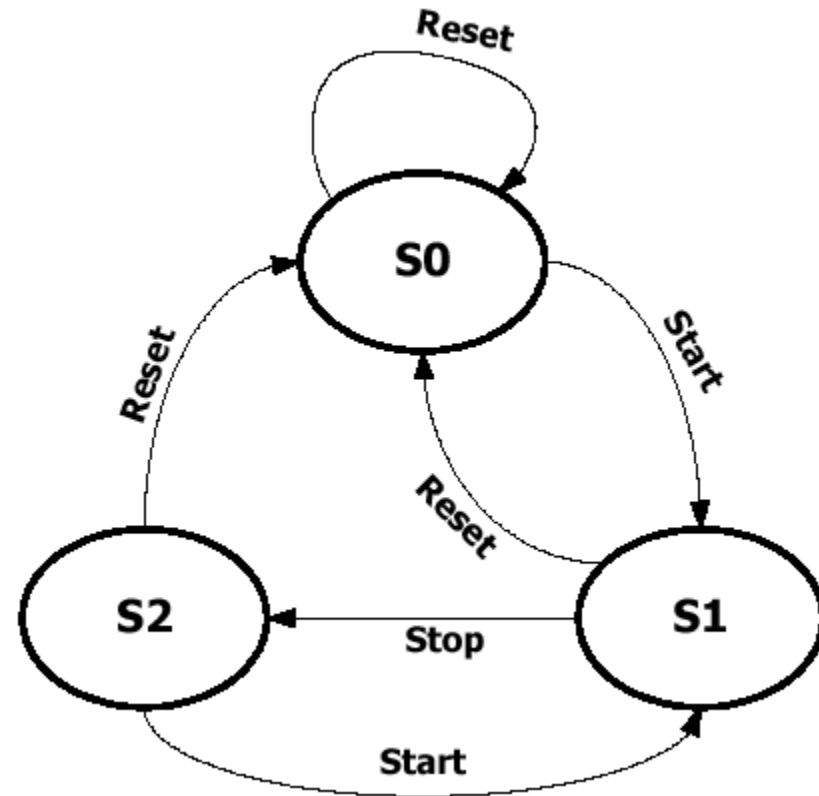


Attributes

- There are many state encoding methods we can use to create Finite State Machines, such as one-hot encoding, Gray encoding, Sequential, Random etc
- The golden rule for the state encoding in designing state machines is, always check the synthesis tools and the hardware device you are using before VHDL coding your state machines

Stop Watch

- Use the state machine, which is shown in Figure, to design a stop watch with the functions below:
 - ❑ a) This stop watch can measure from 0 to 100 seconds with the accuracy of one second.
 - ❑ b) There are three buttons for this stop watch, which are reset, start and stop.
 - ❑ c) The stop watch can be reset to 0 seconds by pressing the reset button.
 - ❑ d) The stop watch output increases by 1 after each second after pressing start until stop is pressed.
 - ❑ Assume a 1Hz clock signal is available for your design.



Stop Watch

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4 entity StopWatch is
5     port(
6         Reset, Clock, Start, Stop: in std_logic;
7         Seconds: out std_logic_vector (6 downto 0));
8 end StopWatch;
9 architecture arch_StopWatch of StopWatch is
10 -- state definitions
11 type stateType is (S0, S1, S2);
12 signal presState: stateType;
13 signal nextState: stateType;
14 signal iCount: std_logic_vector (6 downto 0);
15 signal iCountEnable: std_logic;
16 begin
17     process (Reset, Clock)
18     begin
19         if Clock'event and Clock = '0' then
20             if Reset = '1' then
21                 presState <= S0;
22             else
23                 presState <= nextState;
24             end if;
25             if Reset = '1' then
26                 iCount <= (others=>'0');
27             elsif iCountEnable = '1' then
28                 iCount <= iCount + '1';
29             end if;
30             end if;
31         end process;
32         Seconds <= iCount;
33     process (presState, Start, Stop)
34     begin
35         -- signal defaults
36         iCountEnable <= '0';
37         case presState is
38             when S0 =>
39                 if Start = '1' then
```

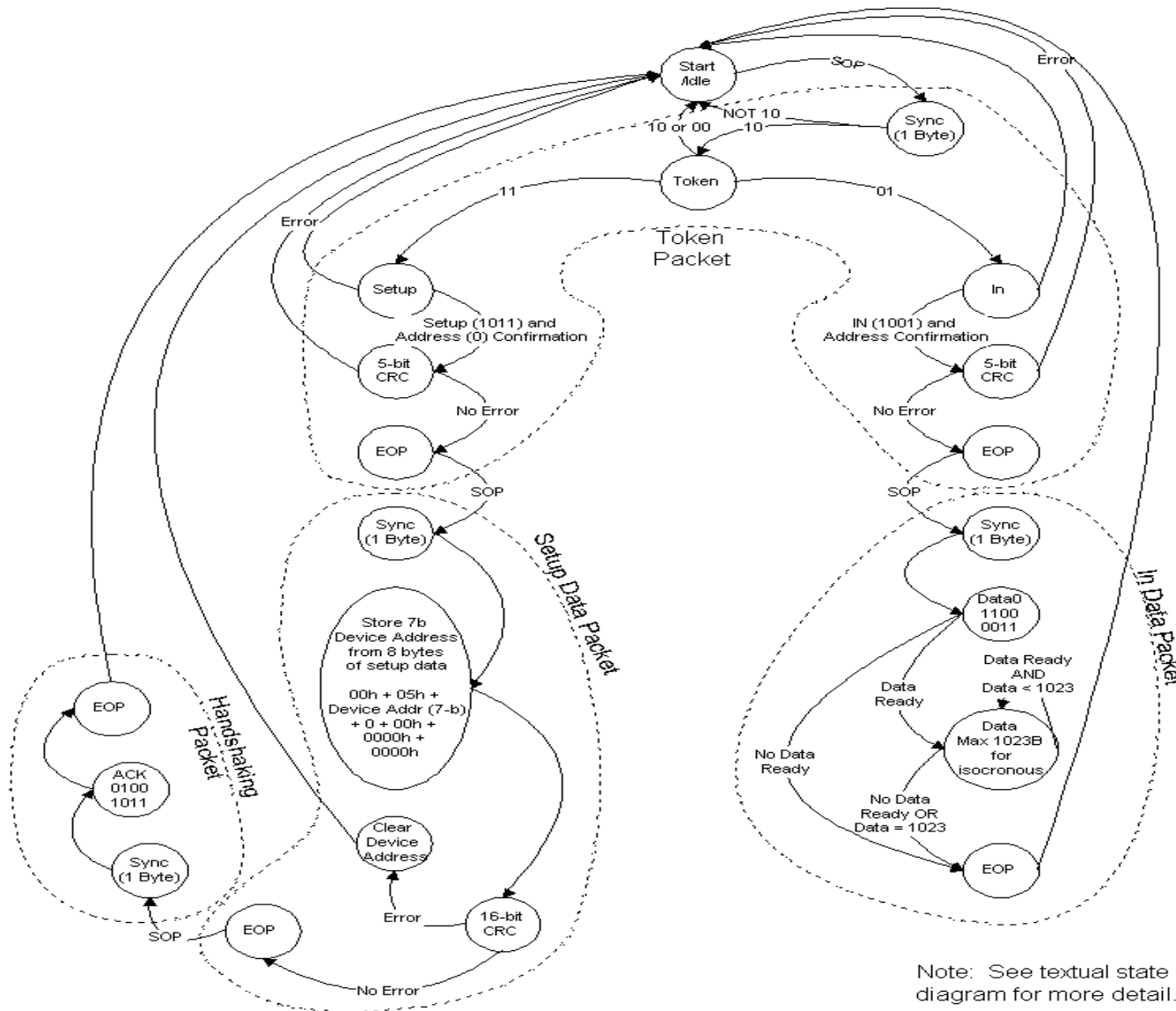
```
40         nextState <= S1;
41         else
42         nextState <= S0;
43         end if;
44         when S1 =>
45             iCountEnable <= '1';
46             if Stop = '1' then
47                 nextState <= S2;
48             else
49                 nextState <= S1;
50             end if;
51         when S2 =>
52             if Start = '1' then
53                 nextState <= S1;
54             else
55                 nextState <= S2;
56             end if;
57         end case;
58     end process;
59 end arch_StopWatch;
```



Intro to State Machine Coding

- What decides the highest clock frequency that a state machine can work?
 - The maximum time taken to calculate the next state or time taken by the combinational circuit.
 - Clock to out time

USB State Machine





END of Lecture 6